

Apostila Teórica: O Guia Definitivo de POO

Módulo: Programação Back-End (PBE-1) - Aulas 29 a 35 **Projeto:** Aetheria Engine API

Bem-vindo ao coração do motor do nosso jogo. Até agora, você trabalhou com Variáveis (**let**), Constantes (**const**), Laços de Repetição (**for**, **while**) e Funções. Tudo isso é o que chamamos de **Programação Estruturada**.

Mas quando construímos sistemas complexos — como o Aetheria Engine —, ter centenas de variáveis espalhadas pelo código se torna um pesadelo. É aqui que entra a **Programação Orientada a Objetos (POO)**.

A POO é um paradigma (um estilo) de programação que tenta aproximar o código do mundo real. No mundo real, você não tem uma "variável hp" voando pelo espaço. Você tem um "Jogador", e esse jogador *possui* um HP.

Vamos entender os 4 pilares desse mundo mágico.

1. Classes, Atributos e Métodos (A Fábrica de Objetos)

Pense na **Classe** como se fosse a planta de uma casa ou a forma de um bolo. Ela não é a casa pronta, e sim as instruções de como construir uma.

Atributos (Características)

São as variáveis que pertencem exclusivamente à classe. No nosso jogo, todo personagem tem um Nome e uma Vida.

Métodos (Ações)

São as funções que a classe sabe executar. Como andar, atacar ou falar.

Veja como criamos a "planta" base de qualquer ser vivo no jogo:

```
// 1. A Classe (O Molde). Por convenção, começa com Letra Maiúscula.
class Entidade {

    // 2. ATRIBUTOS (O que a Entidade TEM)
    nome: string;
    vida: number;

    // O CONSTRUTOR
    constructor(nomeRecebido: string) {
        this.nome = nomeRecebido;
        this.vida = 100;
    }

    // 3. MÉTODOS (O que a Entidade FAZ)
    apresentar() {
        console.log(`Olá, meu nome é ${this.nome} e tenho ${this.vida} de vida!`);
    }
}
```

```
}  
}
```

💡 **O que é o Construtor?** Pense no **constructor** como a esteira de montagem da fábrica. É uma função especial e automática que o TypeScript roda no exato momento em que o objeto está nascendo. É aqui que você obriga o objeto a receber as peças iniciais para funcionar. Se a Entidade não tiver um nome ao nascer, o construtor nem deixa ela ser fabricada!

💡 **A palavra mágica **this**** Quando estamos dentro de uma classe e queremos modificar ou ler um atributo que pertence a ela mesma, precisamos avisar o computador: *"Ei, eu estou falando do atributo DESTA classe, e não de uma variável qualquer perdida por aí"*. O **this** (que significa "isto" ou "este") serve exatamente para isso. Ao escrever **this.nome = nomeRecebido;**, você está dizendo: "Pegue o atributo **nome** desta própria classe e dê a ele o valor que recebemos agora". Sem o **this**, o TypeScript ficaria confuso sem saber de qual nome você está falando!

Instanciando (Dando Vida)

A classe sozinha não faz nada. Para colocar o personagem dentro da memória do computador, usamos a palavra-chave **new**. Ela aciona o **constructor** da nossa classe:

```
let heroi = new Entidade("Arthur");  
heroi.apresentar(); // Acessando o método do objeto!
```

2. Encapsulamento (Protegendo o Core)

No código acima, temos um problema gravíssimo de segurança. Se você digitar **heroi.vida = -50000;**, o TypeScript vai aceitar. O seu jogador estará andando pelo mapa morto!

Na POO, nós não deixamos as propriedades expostas ao mundo. Nós criamos uma "cápsula" ao redor delas. A isso damos o nome de **Encapsulamento**.

Usamos os Modificadores de Acesso para trancar ou destrancar propriedades:

- **public** (Público): É o padrão. Qualquer código, em qualquer arquivo, pode ler e alterar.
- **private** (Privado): Coloca um cadeado. Só o código que está *dentro das chaves { }* daquela classe pode mexer.
- **protected** (Protegido): Um cadeado familiar. Só a classe e as classes que herdarem dela (filhas) podem mexer.

```
class Entidade {  
    public nome: string;  
    private vida: number; // Agora é impossível hackear a vida de fora!
```

Getters e Setters (As Catracas de Segurança)

Se a vida é privada, como o jogo sabe quanto de vida o personagem tem para mostrar na tela? Nós construímos portas de acesso hiper-controladas, chamadas de `get` (pegar) e `set` (definir).

```
// GETTER: Uma fechadura de vidro. Permite apenas LER a vida.
get lerVida(): number {
    return this.vida;
}

// SETTER disfarçado (Método): Aqui nós controlamos como o dano é recebido!
receberDano(dano: number) {
    if (dano > 0) {
        this.vida -= dano;

        // Regra de Ouro: Nunca deixe a vida ficar negativa!
        if (this.vida < 0) {
            this.vida = 0;
        }
    }
}
```

Agora, se alguém tentar `heroi.vida = -50000`, o compilador gritará um erro. Para dar dano no jogador de forma oficial, somos obrigados a usar a catraca de segurança: `heroi.receberDano(50);`.

3. Herança (O Princípio DRY - Don't Repeat Yourself)

Temos a nossa `Entidade`. Mas o jogo terá Vocações diferentes: Magos, Guerreiros e Arqueiros. Todos eles têm nome, vida e sabem receber dano. Seria terrível copiar todo o código da Entidade de novo para o Mago, não é?

A solução se chama **Herança**. Usando a palavra `extends` (estende), criamos uma classe filha que copia magicamente tudo o que a classe pai possui.

A Função `super()`

Há apenas uma regra rígida na Herança: Se o filho nascer, ele precisa ligar para o pai e dizer "*Pai, construa a sua parte do código primeiro*". Fazemos isso chamando a função `super()` dentro do construtor da classe filha.

💡 **Dica de Ouro (Por que o TypeScript exige o `super()`?)** Quando você digita `new Mago("Merlin")`, o computador começa a montar o objeto Mago. Mas o Mago herdou `nome` e `vida` da classe Pai (`Entidade`). Acontece que a classe Mago NÃO SABE como configurar a vida inicial para 100, porque esse código está no construtor do Pai! O `super("Merlin")` é literalmente uma chamada telefônica para o `constructor` da `Entidade`, passando o nome e mandando o Pai preparar o terreno antes do Mago colocar sua própria `mana`.

```
// Mago " copia " a Entidade
class Mago extends Entidade {
    public mana: number;
```

```
    constructor(nome: string) {  
        // Obrigatório: Passa o nome para a Entidade pai configurar!  
        super(nome);  
        this.mana = 100;  
    }  
}
```

Pronto! Um Mago agora tem Nome, Vida, sabe Receber Dano, além de ter sua própria Mana. Tudo isso com pouquíssimas linhas de código.

(Nota: Para que o Mago possa alterar sua própria vida herdada dentro da sua classe, o modificador da **vida** na Entidade deve ser mudado de **private** para **protected**).

4. Polimorfismo (As Múltiplas Formas)

Imagine que a nossa classe Pai (**Entidade**) tenha um método simples de combate:

```
atacar() {  
    console.log(`${this.nome} desferiu um ataque genérico.`);  
}
```

Como o Guerreiro e o Mago herdaram tudo da Entidade, se chamarmos **merlin.atacar()**, ele vai desferir um "ataque genérico". Mas um mago lança feitiços!

O **Polimorfismo** (Poli = Muitas, Morfismo = Formas) nos permite **Sobrescrever** (Override) o método do pai dentro da classe filha.

O nome da ação continua o mesmo (**atacar**), mas a *forma* como ela acontece é diferente para cada vocação:

```
class Guerreiro extends Entidade {  
    atacar() {  
        console.log(`⚔️ ${this.nome} deu uma espadada brutal!`);  
    }  
}  
  
class Mago extends Entidade {  
    atacar() {  
        console.log(`🔥 ${this.nome} lançou uma Bola de Fogo!`);  
    }  
}
```

Resumo do Motor:

Com as **Classes** (Moldes), protegidas por **Encapsulamento** (Cadeados), organizadas por **Herança** (Hierarquia) e flexibilizadas pelo **Polimorfismo** (Ações específicas), temos o poder de construir qualquer sistema Back-End do mundo, desde um jogo de RPG até um sistema bancário complexo.